

Fast On-Device LLM Inference with NPUs

Daliang Xu, Hao Zhang, Liming Yang, Ruiqi Liu, Gang Huang, Mengwei Xu, Xuanzhe Liu
Key Lab of HCST (PKU), MOE; SCS, Peking University, China
Beijing University of Posts and Telecommunications, China

On-Device LLM Inference

Large Language Models (LLMs) are being deployed on mobile/edge devices for:

- Privacy (keep user data on-device).
- Responsiveness (low latency, no network delay).
- Cost reduction (less reliance on cloud compute).

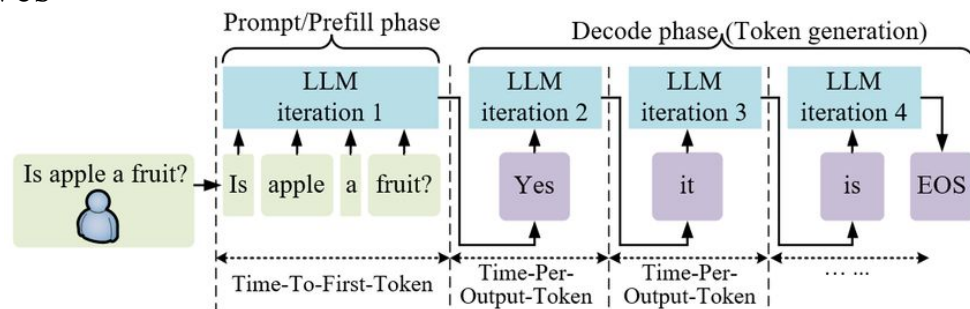
Applications:

- User Interface (UI) Task Automation (8.1 - 40 seconds)
- Automated Message Reply (26.7 seconds)
- Smart Assistants/Voice Commands

Major Bottleneck: High Inference Latency

On-Device LLM Inference on NPUs

- Prefill stage dominates over decode stage (94–99% of total time in UI/email tasks)
- On device LLMs often need long-context understanding (personalized history, screen hierarchy)
- Prefill stage is compute-bound since it involves large matrix multiplications
- Mobile CPUs/GPUs have weak parallelism, not designed for massive vector Ops.



NPUs are now available in mobile SoCs (Qualcomm Hexagon, Google Edge TPU)

- Optimized for integer vector Ops
- Energy efficiency
- High compute throughput
- Low contention

Gaps between LLMs and NPUs

Variable Length Prompts and Graph Overhead

- LLM prefill needs to process variable-length prompts
- Mobile NPUs require fixed-shape compute graphs
- For each new prompt length, graphs must be rebuilt, re-optimized before execution

NPU benefits are negated since overtime dominates runtime

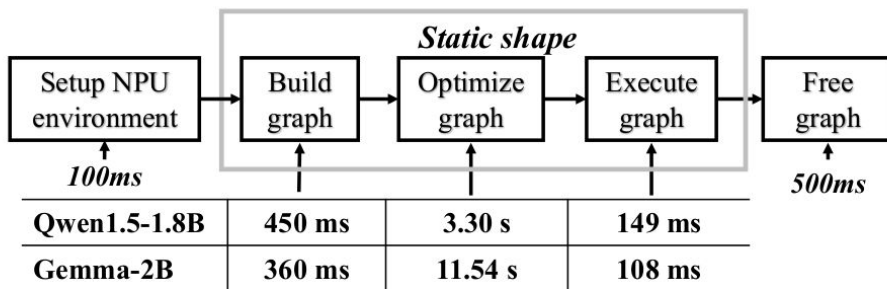


Figure 2. The workflow of executing DNNs on mobile NPUs, with latencies for each procedure on QNN [25].

Gaps between LLMs and NPUs

Quantization mismatch

- LLMs are hard to quantize because of activation outliers
- State-of-the-art LLM quantization methods often use per group quantization method
- NPU are designed for large, uniform, highly parallel MatMul operations

Per-tensor quantization *Per-group quantization*

- NPU-friendly
 - Poor accuracy (outlier distort)
- Accuracy preserved
 - 8.1x - 10.7x slowdown on NPUs

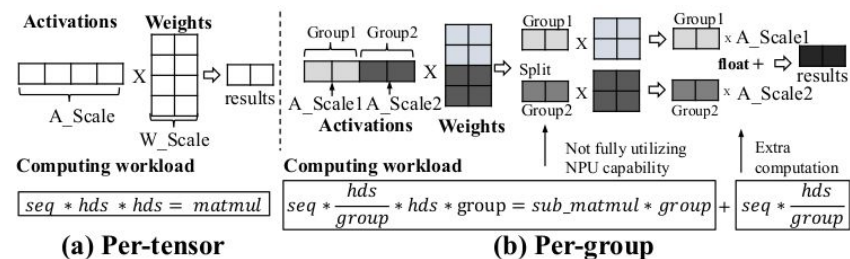
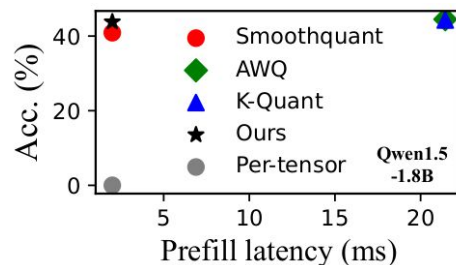
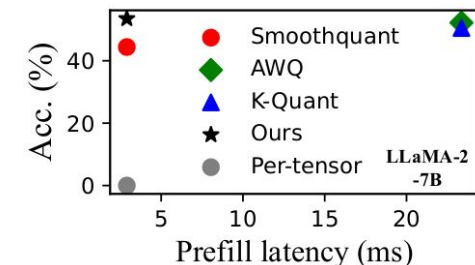


Figure 3. Per-tensor quantization MatMul and per-group quantization MatMul. *seq*, *hds*, *group* represent sequence length, hidden size, and group number, respectively.

Gaps between LLMs and NPUs

Float-heavy operations

- Mobile NPUs are optimized for INT8 acceleration
- But LLM inference still depends on float operations
(Attention, Softmax, scaling, LayerNorm)
- Only linear layers (matmuls) run in INT8 efficiently
- Running float Ops on NPUs is slower
- NPUs can't exploit their vector INT8 engines for these Ops.

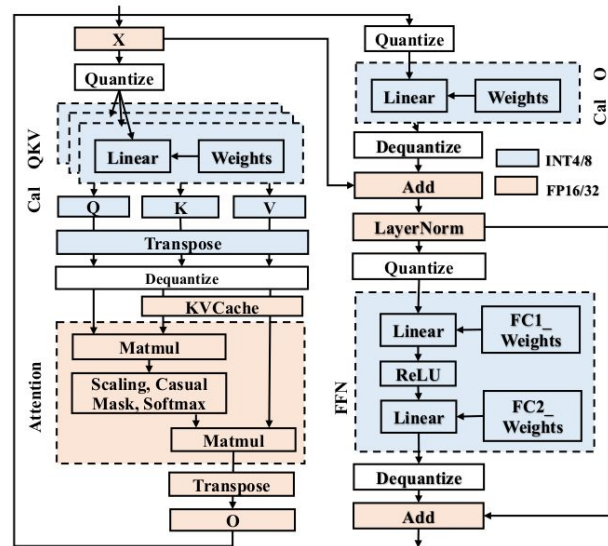


Figure 5. The workflow of quantized on-device LLM inference. Operations shown in blue are computed using INT4/INT8 formats, while those in orange are computed using float data formats.

Overview of llm.npu Design

Design Goal

- Reduce prefill latency and energy consumption for mobile-scale LLMs.
- Enable practical on-device inference on Commercial Off-The-Shelf (COTS) mobile devices.
- Can be integrated into Mobile OS (LLM-as-a-System-Service)

- Maximize execution on NPU (INT8/INT4 acceleration)
- Offload only unavoidable float-heavy Ops to CPU/GPU
- Use new scheduling and quantization strategies to keep NPU busy and avoid stalls

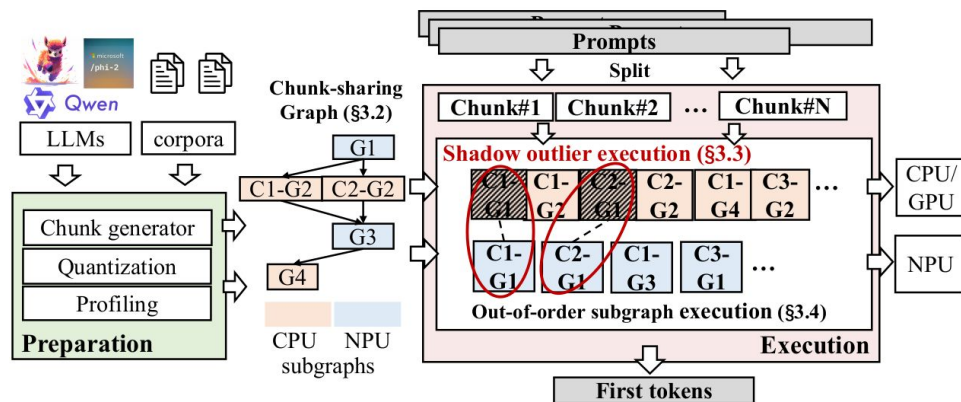


Figure 6. The workflow of llm.npu.

Overview of llm.npu Design

Techniques:

- a) Chunk sharing graph execution**
- b) Shadow outlier execution**
- c) Out-of-Order subgraph execution**

Chunk Sharing Graph Execution

- Fixed-length graphs with padding waste compute resources
- Llm.npu's chunk-based graphs reduce padding but different attention dimensions lead to many graphs in memory, causing high overhead

llm.npu Approach:

- Split prompts into fixed-size chunks (e.g., 256 tokens)
- Pre-build chunk graphs once and reuse during inference
- Static operators: shareable across chunks
- Dynamic operators: chunk specific

- Sharing static operators reduces memory consumption by up to 75%
- Qwen1.5-1.8B saves 7.2 GB for a 1024-token prompt

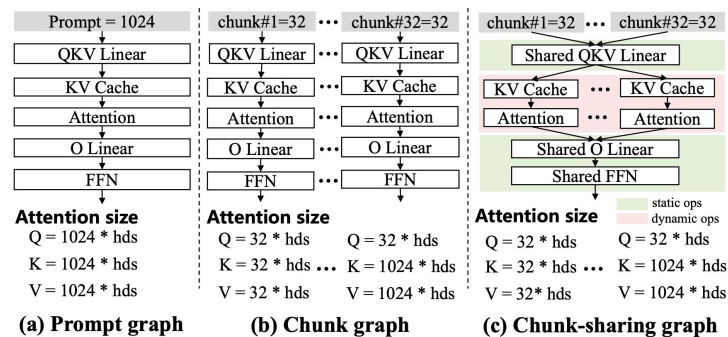


Figure 7. The illustration of prompt graph, chunk graph and chunk-sharing graph. The chunk length is 32.

Shadow Outlier Execution

- Llm.npu use per-tensor MatMul on NPU (fast, INT8)
- Extract outlier channels at runtime into a compact tensor
- Execute outlier contributions on CPU/GPU (float)
- Merge CPU and NPU results to recover full accuracy

$$\begin{aligned} \frac{x}{s} \odot w &= \left\{ \min \left[\max \left(\frac{x}{s}, -127 \right), 128 \right] + \left\lfloor \frac{x}{s} / 128 \right\rfloor \times 128 \right\} \odot w \\ &= \min \left[\max \left(\frac{x}{s}, -127 \right), 128 \right] \odot w \quad \text{on NPU} \\ &+ \text{extract} \left(\left\lfloor \frac{x}{s} / 128 \right\rfloor \times 128 \right) \odot w \quad \text{on CPU} \end{aligned} \quad (1)$$

where x , w , s , $\odot$, and *extract* represent the original float activation, INT8 weights, the quantization scale factor, the MatMul operation, and the function of extracting activation

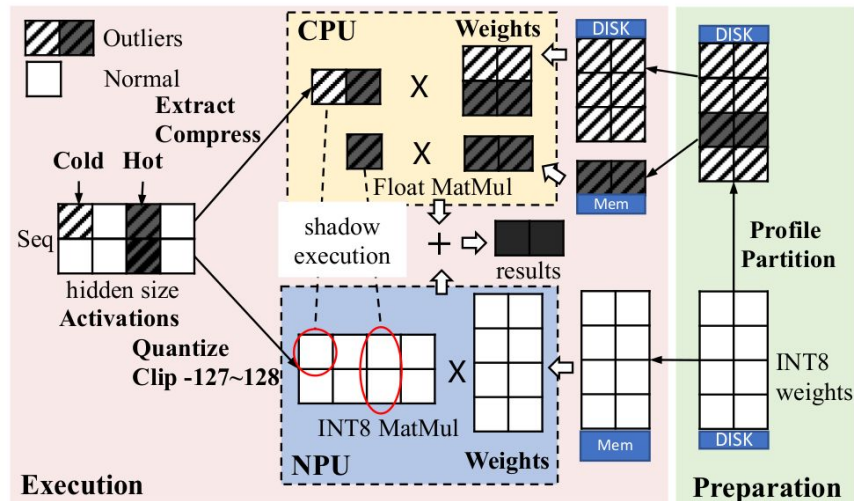


Figure 9. The workflow of shadow outlier execution.

Shadow Outlier Execution

Why it works?

- Outliers are **very sparse** (0.1–0.3% of channels, ~5–15 channels per layer)
- CPU shadow execution is lightweight and overlapped with NPU work
- Achieves both **accuracy** and **NPU efficiency**

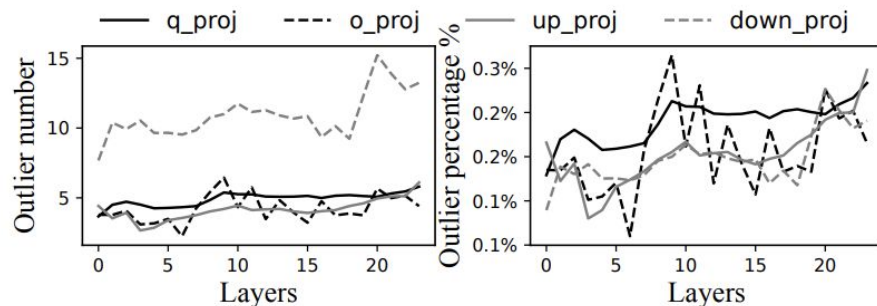


Figure 10. The average number and percentage of outlier channels per layer on Qwen1.5-1.8B model using the wikitext dataset under 2048 inference. **Less than 0.3% channels have outliers during one inference.**

Shadow Outlier Execution - Optimization

- Memory overhead (keeping a copy of all weights in CPU memory)
 - Profiling shows <3% channels cause >80% of outliers
 - llm.npu keeps weights only for “hot channels” in CPU memory
 - Rare “cold channels” are fetched from disk if needed
 - Cuts memory overhead by 34.3% with negligible latency

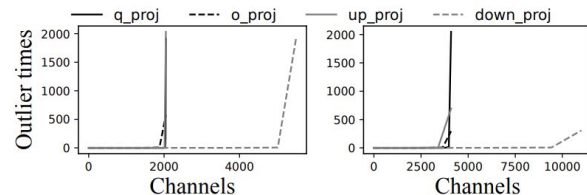


Figure 11. The outlier times per channel on Qwen1.5-1.8B model using the wikitext dataset under 2048 inference. **Less than 3% channels contribute over 80% outliers.**

- Synchronization overhead (result merging adds latency 29.7% and energy cost 20.1%)

- llm.npu prunes outliers from the 85% least important layers (middle layers)
- Keeps only input-near and output-near outliers (most important for accuracy)
- Eliminates most CPU–NPU sync cost

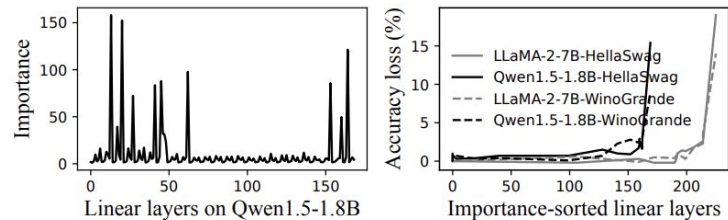


Figure 12. Left: Outlier importance of different layers in Qwen1.5-1.8B model. Right: Relationship between accuracy and pruned layers on HellaSwag and Winograde datasets.

Out-of-order Subgraph Execution

- Simply overlapping float Ops in CPU with NPU causes bubbles
- Llm.npu break model into chunks → subgraphs
- Execute any input-ready subgraph out of order, not strictly chunk by chunk

Dependencies

- Cross-chunk: Attention in chunk i needs outputs from previous chunks
- Intra-chunk: each subgraph depends on the prior subgraph within the same chunk

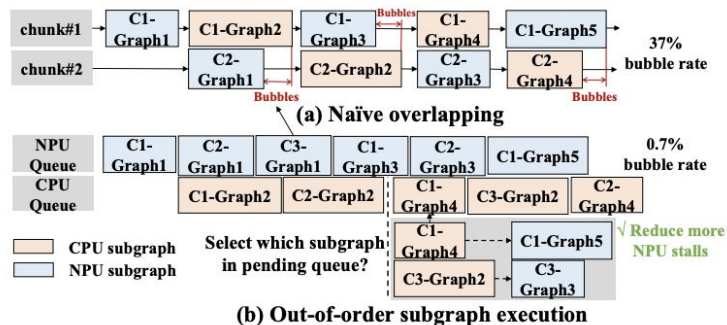


Figure 13. Out-of-order subgraph execution.

Scheduling challenge:(Finding execution order minimizing the total execution time)

- Only one subgraph can run per processor at a time
- Optimal order = NP-hard (like Traveling Salesman)
- Offline strategy impossible (prompt length varies)

Out-of-order Subgraph Execution

Heuristic solution

- Focus on reducing NPU stalls (critical path)
- If subgraph g runs on CPU/GPU:
 - Let S = set of NPU subgraphs unlocked after g completes
 - Longer S execution time $\rightarrow$ better (keeps NPU busy, reduces stalls)
- If subgraph g runs on NPU:
 - Let S = set of CPU subgraphs unlocked after g completes
 - Shorter S execution time $\rightarrow$ better (avoids long CPU bottlenecks)

Scheduling policy:

- Always pick the subgraph with the largest contribution C
- Ensures NPU utilization is maximized and stalls minimized

$$C = \begin{cases} \sum T_i, \forall i \in S \text{ if } g \text{ is on the CPU/GPU} \\ -\sum T_i, \forall i \in S \text{ if } g \text{ is on the NPU} \end{cases}$$

Implementation and Evaluation

Hardware

- Redmi K70 Pro (Snapdragon 8 Gen 3, 24GB RAM)
- Redmi K60 Pro (Snapdragon 8 Gen 2, 16GB RAM)
- OS: Android 13

Models: Qwen1.5-1.8B, Gemma-2B, Phi-2.7B, LLaMA-2-Chat-7B, Mistral-7B

Datasets

- Accuracy evaluation: LAMBADA, HellaSwag, WinoGrande, OpenBookQA, MMLU
- Speed tests: LongBench (retrieval-based), 2WikiMQA, TriviaQA
- UI automation tasks: DroidTask dataset

Evaluation: Prefill Performance

Prefill Speed:

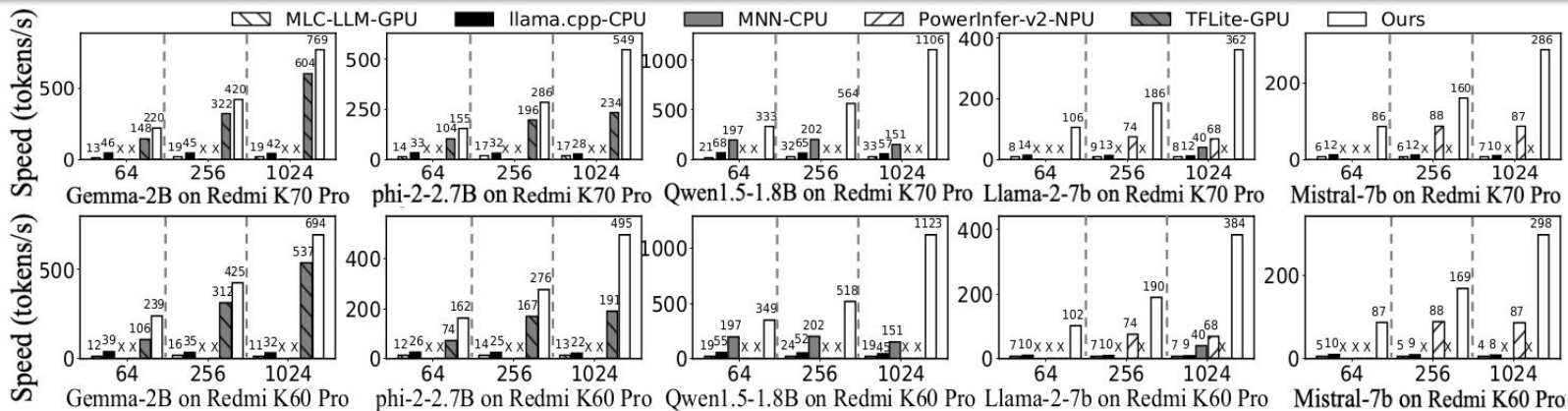


Figure 14. Prefill speed under different prompt lengths on different devices (datasets: Longbench-2wiki-Multi-doc QA).

Prefill Energy Consumption:

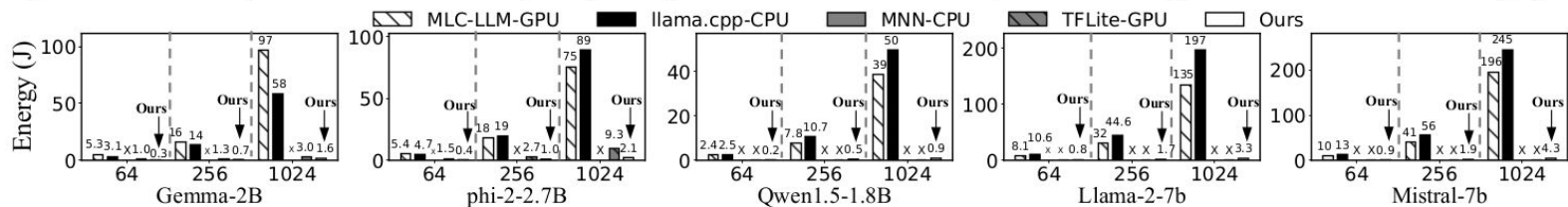


Figure 15. Energy consumption under different prompt lengths on Redmi K60 Pro (datasets: Longbench-2wiki-Multi-doc QA).

Evaluation: End to End Performance

End to End Latency = Prefill Latency + Decode Latency

- Gains are smaller due to shorter prompts since less prefill work compared to email reply tasks
- Even smaller gains because output is long as decode stage dominates because of CPU only decoding

Evaluation: Inference Accuracy

- Dynamic handling of outliers (element-level, CPU float precision)
- Preserve accuracy while keeping NPU efficiency

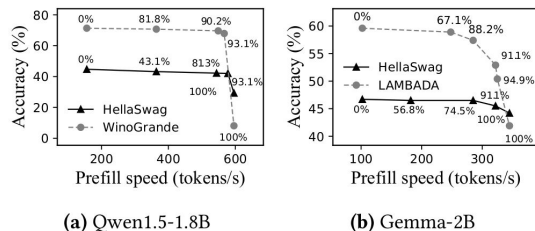


Figure 16. Generation speed and accuracy across various outlier pruning rates. The text positioned above or below the plots represents the corresponding pruning rates.

Accuracy vs Speed Trade-off

0 % pruning → highest accuracy, slower speed

80% pruning → moderate accuracy drop, big speedup

100% pruning → lowest accuracy, fastest speed

Table 6. LLM capability accuracy on 11m. npu and baselines. "SQ": SmoothQuant; "INT8()": LLM.INT8(); "Degrad.": accuracy degradation compared to FP16.

LAMBADA	FP16	SQ	INT8()	K-Quant	Ours	Ours Degrad.
Qwen1.5-1.8B	71.1%	65.6%	71.0%	62.7%	71.7%	+0.6%
Gemma2-2B	59.6%	45.8%	59.2%	56.9%	59.4%	-0.2%
Phi-2-2.7B	72.2%	66.1%	71.7%	59.3%	67.5%	-4.7%
LlaMA-2-7B	87.5%	71.9%	88.0%	15.6%	86.3%	-1.2%
Mistral-7b	84.8%	51.2%	85.3%	23.9%	84.1%	-0.7%
Avg. Degrad.	-	-14.9%	0%	-31.3%	-1.2%	

HellaSwag	FP16	SQ	INT8()	K-Quant	Ours	Ours Degrad.
Qwen1.5-1.8B	43.8%	40.9%	43.5%	44.3%	43.8%	0%
Gemma2-2B	46.5%	43.8%	46.1%	45.4%	47.3%	+0.8%
Phi-2-2.7B	48.2%	46.2%	47.7%	47.6%	46.9%	-1.3%
LlaMA-2-7B	52.8%	44.4%	53.1%	50.5%	53.5%	+0.7%
Mistral-7b	57.4%	44.9%	57.9%	57.0%	57.0%	-0.4%
Avg. Degrad.	-	-5.7%	-0.1%	-0.8%	-0.0%	

WinoGrande	FP16	SQ	INT8()	K-Quant	ours	Ours Degrad.
Qwen1.5-1.8B	58.3%	51.0%	58.2%	59.0%	59.3%	+1.0%
Gemma2-2B	58.3%	54.8%	59.0%	58.5%	59.5%	+1.2%
Phi-2-2.7B	72.2%	68.9%	72.4%	72.5%	70.2%	-2.0%
LlaMA-2-7B	65.2%	56.9%	66.2%	67.4%	65.1%	-0.1%
Mistral-7b	73.5%	59.1%	73.3%	73.5%	73.1%	-0.4%
Avg. Degrad.	-	-7.4%	+0.3%	+0.7%	-0.1%	

OpenBookQA	FP16	SQ	INT8()	K-Quant	ours	Ours Degrad.
Qwen1.5-1.8B	28.8%	23.0%	28.5%	28.0%	26.6%	-2.2%
Gemma2-2B	33.7%	28.0%	34.2%	33.0%	38.4%	+4.7%
Phi-2-2.7B	41.0%	35.9%	40.2%	39.5%	37.7%	-3.3%
LlaMA-2-7B	32.7%	25.0%	32.0%	31.5%	31.1%	-1.6%
Mistral-7b	39.4%	25.6%	39.3%	37.9%	39.3%	-0.1%
Avg. Degrad.	-	-7.6%	-0.3%	-1.1%	-0.5%	

MMLU	FP16	SQ	INT8()	K-Quant	ours	Ours Degrad.
Qwen1.5-1.8B	29.7%	27.9%	29.1%	29.8%	30.8%	+1.1%
Gemma2-2B	35.7%	32.1%	35.1%	35.1%	36.4%	+0.7%
Phi-2-2.7B	35.4%	35.3%	35.6%	35.7%	36.7%	+1.3%
LlaMA-2-7B	37.8%	29.2%	38.1%	34.4%	36.9%	-0.9%
Mistral-7b	42.1%	30.9%	41.4%	42.3%	41.0%	-1.1%
Avg. Degrad.	-	-5.1%	-0.3%	-0.7%	+0.2%	

Evaluation: Memory and Communication

Memory Consumption

- llm.npu uses up to $1.32\times$ more memory than llama.cpp and TFLite
- Overhead mainly from MLLM + QNN frameworks allocating independent activation buffers per operator (for speed)
- Shadow outlier execution adds only 0.6–1% extra memory (tiny float weights)

GPU-NPU Coordination

- Prefill speed: unchanged between CPU-NPU and GPU-NPU (NPU dominates workload, CPU/GPU latency hidden)
- End-to-end latency: GPU-NPU reduces latency by 80–90 ms vs CPU-NPU

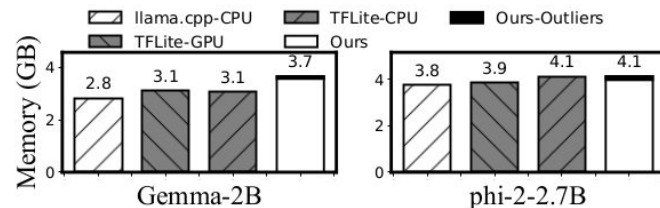
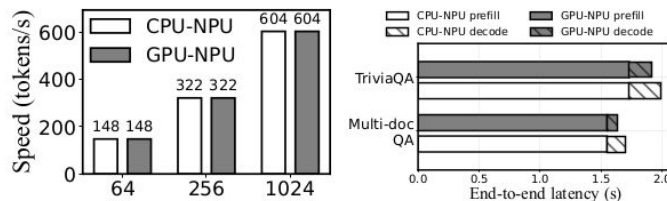


Figure 17. Memory consumption of different baselines (prompt length=512).



(a) Prefill speed under different prompt length **(b)** End-to-end latency on the Longbench dataset

Figure 18. Prefill speed and end-to-end latency comparison between CPU-NPU coordination and GPU-NPU coordination.

Ablation Study

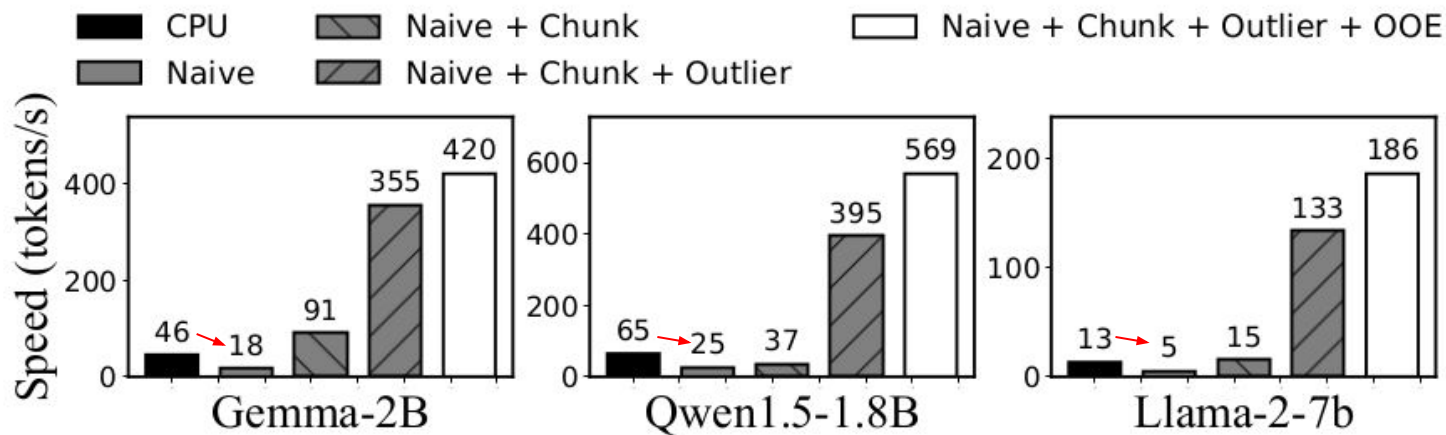


Figure 19. Ablation study of `llm.npu` (prompt length=512).

- Directly offloading LLM prefiling workloads to mobile NPU result in delays due to substantial gap between LLMs and mobile NPUs

Future works

Future Hardware Optimizations

- Dynamic shape-aware optimization: Hardware that adapts to varying input/output shape for LLMs
- Larger NPU data cache: Align cache size with LLM weight sizes for better layer loading
- Mixed Precision compute: Support FP16 * INT8 directly to avoid costly conversions

Llm.npu with Mobile GPUs

- Current: CPU handles float ops and decoding
- Future work: Integrate GPU backend for floating point ops and decoding

Limitations

- Decode latency increases when output length is higher; CPU-only decoding, no acceleration on NPUs
- Not a memory efficient solution
- Limited to INT8 weights
- Only experimented on Qualcomm NPUs
- Does not consider contention when multiple apps use the NPU simultaneously